

TECHCATALYST DE 2026

# Linux CLI & Git collaboration

The terminal is the control surface data engineers work from. Today: shell fluency, quick analysis, automation with cron, and working on a shared codebase without chaos.

The Hartford · TechCatalyst Data Engineering 2026

## Week 2 Day 1

---

The terminal, end to end

Inspect, search, transform

History and automation

Cron scheduling

Git collaboration refresher

---

## DAY 1

# Today's agenda

| Block              | What we do                                               |
|--------------------|----------------------------------------------------------|
| Warm-up            | Why the terminal, first commands (Activity 0)            |
| Mini EDA sprint    | Inspect and analyze files in the shell (Activity 1)      |
| CLI scavenger hunt | Team command hunt with explanations (Activity 2)         |
| Bash + cron        | Turn checks into a script, read schedules (Activity 3)   |
| Group brief        | Choose and defend a terminal strategy (group activity)   |
| Git relay          | Branch, PR, review, merge, resolve conflict (Activity 4) |
| Debrief + stretch  | Share patterns, finish deliverables (Activity 5)         |

# 01

## The terminal

Why it still matters, how to think in it, and what data engineers actually do there.

# Why the terminal?

Data work starts with a question: what arrived, what changed, what failed, what should run next?

- The terminal is a control surface for files, logs, jobs, cloud CLIs, Git, and automation.
- It is text in, text out: precise, scriptable, and repeatable.
- The data engineer's loop: inspect, narrow, transform, verify, automate.
- A notebook or console may be the right tool later. The terminal is usually the fastest first look.

**This week's rule:** If you did it in the terminal once, you can do it again. If you clicked it, you have to remember where you clicked.

---

## SHELL

# Is the terminal still relevant in 2026?

More managed tools, cloud consoles, notebooks, and AI assistants than ever. The terminal still wins on six things.

| What it gives you | Why it matters for data engineering                       |
|-------------------|-----------------------------------------------------------|
| Speed             | Answer a question without opening a notebook or dashboard |
| Repeatability     | Turn a working command into a script, job, or CI step     |
| Auditability      | Commands can be saved, reviewed, committed, rerun         |
| Remote control    | Servers, containers, and runners often have no GUI        |
| AI judgment       | You need fluency to check whether a suggestion is safe    |
| Debugging power   | Logs, paths, permissions, and env usually tell the story  |

## SHELL

# Terminal vs shell, and your options

A **terminal** shows text in and out. A **shell** (Bash, Zsh, PowerShell) interprets what you type.

| System         | Common choices                            | Remember                                                |
|----------------|-------------------------------------------|---------------------------------------------------------|
| Windows        | Windows Terminal, PowerShell, WSL Ubuntu  | PowerShell differs from Bash; WSL is closest to Linux   |
| macOS          | Terminal, iTerm2 (Zsh default)            | Familiar commands, but some flags differ from GNU/Linux |
| Linux / Ubuntu | GNOME Terminal, VS Code terminal (Bash)   | Closest match to servers, containers, and cloud shells  |
| Cloud          | AWS CloudShell, Google Cloud Shell, Azure | Credentials and CLIs are often preconfigured            |

Know which shell you are in before you copy a command. Today we use Ubuntu with Bash.

---

## SHELL

# The cloud angle: cloud shells

- A cloud shell is a browser-based terminal that runs next to your cloud resources.
- Credentials and CLIs (aws, gcloud, az) are already set up, so you skip local install and login friction.
- Same skills transfer: the commands you learn today work there too.
- Watch the limits: ephemeral storage, session timeouts, and quotas vary by provider.

| Provider     | Cloud shell | Typical use                            |
|--------------|-------------|----------------------------------------|
| AWS          | CloudShell  | Run aws CLI, poke at S3, quick scripts |
| Google Cloud | Cloud Shell | Run gcloud / bq, manage buckets        |
| Azure        | Cloud Shell | Run az CLI against resources           |

## SHELL

# Navigation & file operations

| Command                        | What it does                   | Flags you will actually use                      |
|--------------------------------|--------------------------------|--------------------------------------------------|
| <code>pwd / ls / cd</code>     | Where am I, what is here, move | <code>ls -lh</code> (long, human sizes)          |
| <code>mkdir / rmdir</code>     | Create / remove directories    | <code>mkdir -p a/b/c</code> (parents)            |
| <code>cp / mv / rm</code>      | Copy, move or rename, delete   | <code>cp -r, rm -i</code> (confirm)              |
| <code>cat / head / tail</code> | Print file, first / last lines | <code>tail -f</code> (follow a live log)         |
| <code>less</code>              | Page through big files         | <code>q</code> to quit, <code>/</code> to search |
| <code>wc / du</code>           | Count lines, measure size      | <code>wc -l data.csv, du -sh dir/</code>         |

Muscle memory comes from typing. Purpose comes from knowing when the terminal is the right tool.



## Command history: stop retyping

```
history          # list recent commands, numbered
history | grep grep # find that command you ran earlier
!!              # rerun the last command
!457            # rerun command number 457
!$              # reuse the last argument of the last command
sudo !!         # rerun the last command with sudo
```

- Ctrl-R searches history as you type. Press it again to step back further.
- Up arrow walks recent commands. History persists across sessions.
- If a command was worth running twice, it is worth saving in a script.

---

## SHELL

# Pipes & redirection: composing tools

```
# count unique buyers in a CSV (skip the header row)
tail -n +2 orders_sample.csv | cut -d',' -f2 | sort | uniq | wc -l

# save matching log lines to a file
grep -i "error" pipeline.log > errors.txt

# append a run marker to a log (>> appends, > overwrites)
echo "done $(date)" >> run_history.log
```

A pipe (|) sends one command's output into the next. That is the same staged thinking as a data pipeline.

---

## SHELL

# Finding things: grep & find

```
# grep: search INSIDE files
grep -i "timeout" pipeline.log      # case-insensitive
grep -r "TODO" src/                 # recurse through a folder
grep -c "ERROR" pipeline.log        # count matches

# find: search FOR files
find . -name "*.parquet"            # by name pattern
find data/ -size +100M               # bigger than 100 MB
find logs/ -mtime -1                 # modified in the last 24h
```

grep looks inside files; find looks for files. Interviewers love asking you to combine them.

---

## SHELL

# Beyond navigation: terminal EDA

Before Python or SQL, the shell answers first-pass data questions fast.

```
head -n 5 orders.csv          # what do the columns look like
wc -l orders.csv              # how many rows arrived
cut -d',' -f3 orders.csv | sort | uniq -c | sort -nr    # top values
awk -F',' 'NR>1 {s+=$5} END {print s}' orders.csv      # sum a column
```

Terminal EDA is a fast first pass. It tells you whether the file is sane before you spend time loading it.

## SHELL

# Where data engineers use the terminal

| Area         | What you do                          | Example moves                    |
|--------------|--------------------------------------|----------------------------------|
| File triage  | Check what arrived and how big       | ls -lh, du -sh, find, wc -l      |
| Data EDA     | Preview rows, count, find bad values | head, cut, sort, uniq, awk       |
| Log analysis | Find errors, watch jobs live         | grep, tail -f, uniq -c           |
| Automation   | Turn sequences into scripts          | Bash variables, loops, redirects |
| Scheduling   | Describe when jobs run               | cron, later Airflow / Composer   |
| Git          | Branch, review, merge, recover       | git switch, status, pull, push   |

Same five-move loop everywhere: inspect, narrow, transform, verify, automate.

## From commands to scripts

```
#!/usr/bin/env bash
DATA_DIR="./operations_playground/data"
for f in "$DATA_DIR"/*.csv; do
    rows=$(wc -l < "$f")
    echo "$f has $rows rows"
    if [ "$rows" -lt 2 ]; then echo "  WARNING: looks empty"; fi
done
```

- Variables, a loop, and a condition turn repeated checks into one command.
- A script is easy to review, commit, rerun, and schedule. A remembered command is none of those.

---

## SHELL

# Scheduling with cron

```
# minute hour day-of-month month day-of-week command
30 2 * * * bash /opt/jobs/daily_load.sh
0 8 * * 1 bash /opt/jobs/weekly_report.sh
*/15 * * * * bash /opt/jobs/poll_api.sh

crontab -e # edit your schedule      crontab -l # list it
```

- Read them aloud: daily at 2:30 AM, Mondays at 8:00 AM, every 15 minutes.
- Production habit: absolute paths, redirect output to a log, make jobs safe to rerun (idempotent).
- cron is the simplest scheduler. Airflow and Cloud Composer come later.

## SHELL

# Terminal safety in the AI era

AI can suggest a command. It cannot see your folder, files, credentials, or permissions. You verify before you run.

| Check    | Ask yourself                                            |
|----------|---------------------------------------------------------|
| Location | What folder am I in? Run pwd.                           |
| Target   | What will this touch? Run ls or find first.             |
| Effect   | Does it read, write, move, overwrite, or delete?        |
| Scope    | One file, many files, or everything recursively?        |
| Recovery | If this goes wrong, can I undo it from Git or a backup? |

**AI-Free Zone:** Weeks 1 to 4 are an AI-Free Zone for writing code. Use man, tldr, explainshell, and your team, not an AI assistant.



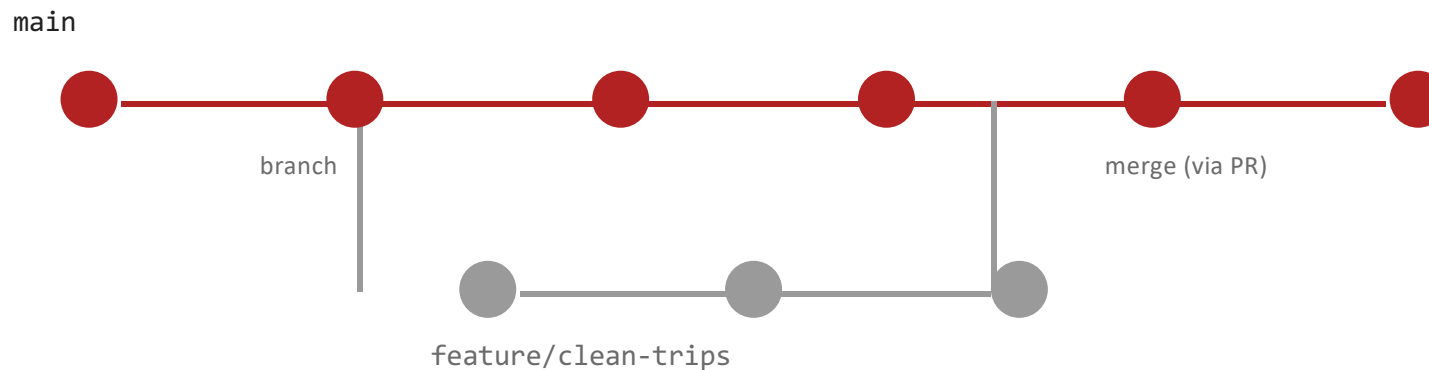
# 02

## Git collaboration

A refresher from Week 1 Day 3, then straight into a team relay: branch, PR, review, merge, resolve.

## GIT

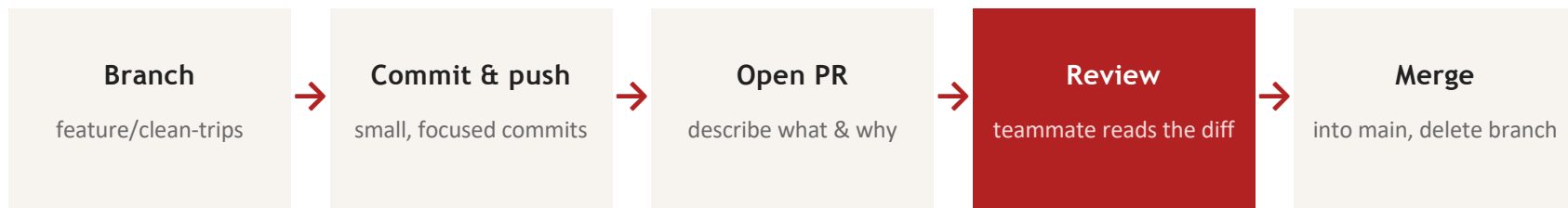
# Branches and the daily loop



**The daily loop:** `git status` → `git add` → `git commit -m` → `git push`

- `main` stays releasable. Work happens on short-lived feature branches (`git switch -c feature/clean-trips`).
- `git pull origin main` keeps your branch current; `git push -u` publishes it the first time.

# The pull request workflow



- The PR is where engineering culture lives: review comments, suggestions, approvals.
- Review etiquette: critique code, not people; ask questions; approve when it is better, not perfect.
- From today, all team submissions flow through PRs, including this afternoon's relay.

## Anatomy of a merge conflict

```
<<<<<<< HEAD
fare_threshold = 100
=====
fare_threshold = 250
>>>>>>> main
```

- Conflicts are normal. Two people changed the same lines.
- Fix on your branch: choose or combine versions, delete the marker lines, save, add, commit.
- Read both sides first. The right answer is sometimes both, sometimes neither.

## LABS

# What you will build today

| Deliverable                                            | From           |
|--------------------------------------------------------|----------------|
| eda_notes.md: file, store, and log observations        | Activity 1     |
| cli_hunt.md: 12 commands with answers and explanations | Activity 2     |
| summarize_hunt.sh: reusable file, row, and log checks  | Activity 3     |
| Cron annotations for 4 job patterns                    | Activity 3     |
| Terminal operations decision brief (2-min present)     | Group activity |
| Git relay repo: merged PRs and resolved conflicts      | Activity 4     |
| Exit ticket                                            | End of day     |

---

## RECAP

# Key takeaways

1

The terminal is a control surface: inspect, narrow, transform, verify, automate.

2

It is still essential in 2026 for speed, repeatability, remote work, and checking AI suggestions.

3

Pipes compose small tools; grep searches inside files, find searches for files; cron schedules them.

4

Verify before you run: location, target, effect, scope, recovery.

5

Git collaboration is branch, PR, review, merge, and resolving conflicts without breaking main.